Analysis of Algorithm (CSN13400)
ASSIGNMENT-7

Name : Aayansh Niranjan Date :01/10/2026
Reg_No.: 20253004

Q1. In modern operating systems, job scheduling is crucial for ensuring that critical system tasks are executed in a timely manner. Some processes, like handling interrupts or urgent system maintenance tasks, need higher priority than regular user processes. Design and implement a Priority Queue in C to manage system processes based on their priority levels. Higher priority processes should be executed first, while processes with the same priority are handled in a first-come, first-served (FIFO) manner. Implement the system with at least 10 processes, each having a unique ID and a priority value (10 being the highest, 1 being the lowest). Some examples of processes are: Interrupt Handling, System Maintenance, User Applications, and Background Tasks.
Your program should support the following operations:

Insert Process: Add a new process to the queue with a priority level and process ID.

Remove Highest Priority Process: Remove and return the process with the highest priority for execution.

Peek at Highest Priority Process: View the process with the highest priority without removing it.

Change Process Priority: Modify the priority of a process already in the queue.

A1. CODE:

```
#include <stdio.h>
#include <string.h>

#define N 100

typedef struct{
    int id,priority;
    char name[30];
} Process;

Process q[N];
int n=0;

void insert(int id,int p,char name[]){
    q[n].id=id;
    q[n].priority=p;
    strcpy(q[n].name,name);
    n++;
}

int highest(){
    int k=0;
    for(int i=1;i<n;i++)
        if(q[i].priority>q[k].priority)
            k=i;
    return k;
}
```

```
void peek(){
    if(!n) return;
    int k=highest();
    printf("Highest: ID=%d Priority=%d %s\n",
           q[k].id,q[k].priority,q[k].name);
}

void removeProcess(){
    if(!n) return;
    int k=highest();
    printf("Executed: ID=%d Priority=%d %s\n",
           q[k].id,q[k].priority,q[k].name);
    for(int i=k;i<n-1;i++) q[i]=q[i+1];
    n--;
}

void changePriority(int id,int p){
    for(int i=0;i<n;i++)
        if(q[i].id==id){
            q[i].priority=p;
            return;
        }
}

void display(){
    for(int i=0;i<n;i++)
        printf("%d\t%d\t%s\n",q[i].id,q[i].priority,q[i].name);
}

int main(){
    insert(1,10,"Interrupt Handling");
    insert(2,8,"System Maintenance");
    insert(3,5,"User Application");
    insert(4,3,"Background Task");
    insert(5,8,"File System");
    insert(6,6,"Network Service");
    insert(7,4,"Backup Task");
    insert(8,5,"Database");
    insert(9,2,"Logging");
    insert(10,1,"Idle Task");
    printf("Initial Queue:\n");
    display();
    peek();
    changePriority(4,9);
    printf("\nAfter Priority Change:\n");
    display();
    removeProcess();
    printf("\nAfter Removal:\n");
    display();
    return 0;
}
```

OUTPUT:

```
Initial Queue:
1       10      Interrupt Handling
2       8       System Maintenance
3       5       User Application
4       3       Background Task
5       8       File System
6       6       Network Service
7       4       Backup Task
8       5       Database
9       2       Logging
10      1       Idle Task
Highest: ID=1 Priority=10 Interrupt Handling

After Priority Change:
1       10      Interrupt Handling
2       8       System Maintenance
3       5       User Application
4       9       Background Task
5       8       File System
6       6       Network Service
7       4       Backup Task
8       5       Database
9       2       Logging
10      1       Idle Task
Executed: ID=1 Priority=10 Interrupt Handling

After Removal:
2       8       System Maintenance
3       5       User Application
4       9       Background Task
5       8       File System
6       6       Network Service
7       4       Backup Task
8       5       Database
9       2       Logging
10      1       Idle Task
```

Q2. In real-world applications such as data storage, file transfer, and communication systems, efficient data compression is critical for reducing storage space and bandwidth usage. One of the most widely used algorithms for lossless data compression is Huffman Coding, which is based on a greedy approach to minimize the average length of encoding symbols. Design and implement a Huffman Coding algorithm in C to compress a given text file. Your implementation should build a binary tree based on the frequency of characters in the input, and then generate an optimal prefix-free binary code for each character to achieve maximum compression. Your program should be able to read from a text file of any size. For demonstration, use a sample text file containing English sentences or paragraphs.

Output: Display the original text, the Huffman codes for each character, and the compressed binary string. Compare and display the compression ratio.

A2. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node{
   unsigned char c;
   int f;
   struct Node *l,*r;
} Node;

Node *heap[256];
int hn=0;
char code[256][256];

void push(Node *x){
   int i=hn++;
   while(i && x->f<heap[(i-1)/2]->f){
      heap[i]=heap[(i-1)/2];
      i=(i-1)/2;
   }
   heap[i]=x;
}

Node* pop(){
   Node *x=heap[0],*y=heap[--hn];
   int i=0;
   while(2*i+1<hn){
      int j=2*i+1;
      if(j+1<hn && heap[j+1]->f<heap[j]->f) j++;
      if(y->f<=heap[j]->f) break;
      heap[i]=heap[j];
      i=j;
   }
   if(hn) heap[i]=y;
   return x;
}
```

```
void makeCode(Node *r,char *s,int d){
   if(!r->l&&!r->r){
      s[d]='\0';
      strcpy(code[r->c],s);
      return;
   }
   s[d]='0'; makeCode(r->l,s,d+1);
   s[d]='1'; makeCode(r->r,s,d+1);
}

Node* node(unsigned char c,int f,Node *l,Node *r){
   Node *x=malloc(sizeof(Node));
   x->c=c;x->f=f;x->l=l;x->r=r;
   return x;
}

int main(){
   FILE *fp=fopen("input.txt","rb");
   if(!fp){printf("File not found\n");return 0;}

   int cap=1024,n=0,f[256]={0};
   unsigned char *a=malloc(cap);
   int ch;

   while((ch=fgetc(fp))!=EOF){
      if(n==cap) a=realloc(a,cap*=2);
      a[n++]=ch;
      f[ch]++;
   }
   fclose(fp);

   printf("Original Text:\n");
   for(int i=0;i<n;i++) putchar(a[i]);

   for(int i=0;i<256;i++)
      if(f[i]) push(node(i,f[i],NULL,NULL));

   while(hn>1){
      Node *x=pop(),*y=pop();
      push(node(0,x->f+y->f,x,y));
   }

   char s[256];
   makeCode(heap[0],s,0);

   printf("\n\nHuffman Codes:\n");
   for(int i=0;i<256;i++)
      if(f[i])
         printf("%c : %s\n",i,code[i]);

   printf("\nCompressed Binary:\n");
   long bits=0;
   for(int i=0;i<n;i++){
      printf("%s",code[a[i]]);
      bits+=strlen(code[a[i]]);
   }

   printf("\n\nOriginal Bits: %ld",n*8L);
   printf("\nCompressed Bits: %ld",bits);
   printf("\nCompression Ratio: %.2f%%\n",
        100.0*bits/(n*8));

   free(a);
   return 0;
}
```

OUTPUT:

INPUT.TXT

```
Huffman coding is a greedy algorithm used for data compression.
It assigns shorter codes to frequent characters.
```

```
Original Text:
Huffman coding is a greedy algorithm used for data compression.
It assigns shorter codes to frequent characters.

Huffman Codes:

 : 0101010
  : 100
. : 00110
H : 0101011
I : 1011010
a : 1100
c : 0100
d : 11110
e : 1101
f : 10111
g : 01101
h : 01011
i : 11111
l : 1011011
m : 01100
n : 0010
o : 1010
p : 1011000
q : 1011001
r : 1110
s : 000
t : 0111
u : 00111
y : 010100

Compressed Binary:
0101011001111011110111011001100001010001001010111011111100100111011001111110001001100100011011110111011101111100101001001100101101101101101111011111011101011001000011100011101110010000111011010000011111101000100110010101011010011110011000000001111101101001000010000001011101011100111110111101000100011110101001011111101101101100100111110100100111100010001011110011101100010001111101111000000110

Original Bits: 896
Compressed Bits: 472
Compression Ratio: 52.68%
```

Q3.Implementation and Analysis of Quick Sort versus Randomised Quick Sort in C for the Worst-Case Analysis Using GNU Plot.

A3. CODE:

```
#include <stdio.h>

typedef struct{
   int id,start,end;
} Job;

void sort(Job a[],int n){
   for(int i=0;i<n-1;i++)
      for(int j=0;j<n-i-1;j++)
         if(a[j].end>a[j+1].end){
            Job t=a[j];
            a[j]=a[j+1];
            a[j+1]=t;
         }
}

int main(){
   Job a[]={
      {1,1,4},{2,3,5},{3,0,6},{4,5,7},{5,3,9},
      {6,5,9},{7,6,10},{8,8,11},{9,8,12},{10,2,14}
   };
   int n=10,last=-1,count=0;

   sort(a,n);

   printf("Selected Meetings:\n");

   for(int i=0;i<n;i++)
      if(a[i].start>=last){
         printf("Job %d: %d-%d\n",
               a[i].id,a[i].start,a[i].end);
         last=a[i].end;
         count++;
      }

   printf("Maximum Meetings = %d\n",count);
   return 0;
}
```

OUTPUT:

```
Selected Meetings:
Job 1: 1-4
Job 4: 5-7
Job 8: 8-11
Maximum Meetings = 3
```